Simple Object-Oriented Programming (OOP) Tutorial in Python

Building a Calculator with Base Conversion and Colorful Menus

In this tutorial, you'll learn the basics of OOP in Python by building a calculator step by step. We'll start with a procedural approach (functions + menu), then refactor into a `Calculator` class, add a **base property** (decimal/octal/binary), and finally create a `Menu` class that uses **colorama** to display colored output.

Table of Contents

1. Prerequisites
2. Step 1 – Procedural Calculator
3. Step 2 – Calculator Class
4. Step 3 – Adding a Base Property
5. Step 4 – Menu Class with Colorama
6. Full Code
7. How to Run

Prerequisites

- Python 3 installed
- Install `colorama` (if not already installed):

```
pip install colorama
```

Step 1 – Procedural Calculator

We first write four basic arithmetic functions and a simple text menu. The user selects an operation and enters two numbers.

```python
# step1_procedural.py

def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

def multiply(a, b):
    return a * b
```

```python
def divide(a, b):
    if b == 0:
        return "Error: Division by zero"
    return a / b

def show_menu():
    print("\n--- Calculator Menu ---")
    print("1. Add")
    print("2. Subtract")
    print("3. Multiply")
    print("4. Divide")
    print("5. Exit")

def main():
    while True:
        show_menu()
        choice = input("Choose an option (1-5): ")
        if choice == '5':
            print("Goodbye!")
            break
        if choice not in ('1','2','3','4'):
            print("Invalid choice. Try again.")
            continue

        try:
            num1 = float(input("Enter first number: "))
            num2 = float(input("Enter second number: "))
        except ValueError:
            print("Please enter valid numbers.")
            continue

        if choice == '1':
            result = add(num1, num2)
            print(f"Result: {num1} + {num2} = {result}")
        elif choice == '2':
            result = subtract(num1, num2)
            print(f"Result: {num1} - {num2} = {result}")
        elif choice == '3':
            result = multiply(num1, num2)
            print(f"Result: {num1} * {num2} = {result}")
        elif choice == '4':
            result = divide(num1, num2)
            print(f"Result: {num1} / {num2} = {result}")

if __name__ == "__main__":
    main()
```

What we learned:
Functions keep code organised, but they don't hold state. For a calculator that could remember a base or last result, a class is better.

Step 2 – Calculator Class

We wrap the same operations inside a `Calculator` class. The class can store a **current value** (optional) and group methods logically.

```python
# step2_class.py

class Calculator:
    """A simple calculator with basic arithmetic operations."""

    def add(self, a, b):
        return a + b

    def subtract(self, a, b):
        return a - b

    def multiply(self, a, b):
        return a * b

    def divide(self, a, b):
        if b == 0:
            raise ValueError("Cannot divide by zero")
        return a / b

def main():
    calc = Calculator()          # create an instance
    while True:
        print("\n--- Calculator Menu ---")
        print("1. Add")
        print("2. Subtract")
        print("3. Multiply")
        print("4. Divide")
        print("5. Exit")
        choice = input("Choose: ")
        if choice == '5':
            break
        if choice not in ('1','2','3','4'):
            print("Invalid choice")
            continue

        try:
            x = float(input("First number: "))
            y = float(input("Second number: "))
        except ValueError:
            print("Invalid number")
            continue

        try:
            if choice == '1':
                result = calc.add(x, y)
                print(f"{x} + {y} = {result}")
            elif choice == '2':
```

```
                result = calc.subtract(x, y)
                print(f"{x} - {y} = {result}")
            elif choice == '3':
                result = calc.multiply(x, y)
                print(f"{x} * {y} = {result}")
            elif choice == '4':
                result = calc.divide(x, y)
                print(f"{x} / {y} = {result}")
        except ValueError as e:
            print(f"Error: {e}")

if __name__ == "__main__":
    main()
```

OOP concept:
A class groups data (attributes) and behaviour (methods). Here we use only methods, but we'll add data in the next step.

Step 3 – Adding a Base Property

We extend the `Calculator` class with a **base** property that determines how numbers are displayed (decimal, octal, or binary).
We'll use the `@property` decorator to control access to the `_base` attribute.

```
# step3_base_property.py

class Calculator:
    def __init__(self, base="decimal"):
        """
        base can be 'decimal', 'octal', or 'binary'.
        """
        self._base = base   # internal attribute (convention: _name)

    @property
    def base(self):
        """Getter for base."""
        return self._base

    @base.setter
    def base(self, new_base):
        """Setter: validate allowed bases."""
        if new_base in ("decimal", "octal", "binary"):
            self._base = new_base
        else:
            raise ValueError("Base must be 'decimal', 'octal', or 'binary'")

    def _format_result(self, value):
        """Format the result according to the current base."""
        if self.base == "decimal":
            return str(value)
```

```
        elif self.base == "octal":
            # Convert to integer then to octal string (without '0o' prefix)
            return oct(int(value))[2:]
        elif self.base == "binary":
            return bin(int(value))[2:]
        return str(value)

    # Arithmetic methods (unchanged)
    def add(self, a, b):
        result = a + b
        return self._format_result(result)

    def subtract(self, a, b):
        result = a - b
        return self._format_result(result)

    def multiply(self, a, b):
        result = a * b
        return self._format_result(result)

    def divide(self, a, b):
        if b == 0:
            raise ValueError("Cannot divide by zero")
        result = a / b
        # For division, we keep float; octal/binary only for integers.
        # We'll convert only if result is integer-ish.
        if result.is_integer():
            result = int(result)
            return self._format_result(result)
        else:
            return str(result)   # decimal format for non-integers

def main():
    calc = Calculator()
    # Let user choose base first
    print("Choose output base:")
    print("1. Decimal\n2. Octal\n3. Binary")
    base_choice = input("Enter 1-3: ")
    if base_choice == '2':
        calc.base = "octal"
    elif base_choice == '3':
        calc.base = "binary"
    else:
        calc.base = "decimal"

    while True:
        print(f"\n--- Calculator (Base: {calc.base}) ---")
        print("1. Add\n2. Subtract\n3. Multiply\n4. Divide\n5. Change base\n6.
Exit")
        choice = input("Choose: ")
        if choice == '6':
            break
        if choice == '5':
            new_base = input("New base (decimal/octal/binary): ")
```

```
            try:
                calc.base = new_base
                print(f"Base changed to {calc.base}")
            except ValueError as e:
                print(e)
            continue
        if choice not in ('1','2','3','4'):
            print("Invalid")
            continue

        try:
            x = float(input("First number: "))
            y = float(input("Second number: "))
        except ValueError:
            print("Invalid number")
            continue

        try:
            if choice == '1':
                res = calc.add(x, y)
                print(f"{x} + {y} = {res}")
            elif choice == '2':
                res = calc.subtract(x, y)
                print(f"{x} - {y} = {res}")
            elif choice == '3':
                res = calc.multiply(x, y)
                print(f"{x} * {y} = {res}")
            elif choice == '4':
                res = calc.divide(x, y)
                print(f"{x} / {y} = {res}")
        except ValueError as e:
            print(f"Error: {e}")

if __name__ == "__main__":
    main()
```

What we learned about properties:

- `@property` lets you define a getter that is accessed like an attribute (`calc.base`).
- `@base.setter` lets you validate the value before changing an internal variable.
- Encapsulation: internal `_base` is protected; users interact via the property.

Step 4 – Menu Class with Colorama

Now we create a separate `Menu` class that handles user interaction and displays colourful output using the `colorama` library. We'll also integrate the `Calculator` class.

```
# step4_menu_class.py

import colorama
```

```
from colorama import Fore, Back, Style

# Initialize colorama (works on Windows, macOS, Linux)
colorama.init(autoreset=True)

class Calculator:
    """Same as before, but we simplify formatting for clarity."""
    def __init__(self, base="decimal"):
        self._base = base

    @property
    def base(self):
        return self._base

    @base.setter
    def base(self, new_base):
        if new_base in ("decimal", "octal", "binary"):
            self._base = new_base
        else:
            raise ValueError("Base must be 'decimal', 'octal', or 'binary'")

    def _format(self, value):
        if self.base == "decimal":
            return str(value)
        elif self.base == "octal":
            return oct(int(value))[2:]
        elif self.base == "binary":
            return bin(int(value))[2:]
        return str(value)

    def add(self, a, b):
        return self._format(a + b)

    def subtract(self, a, b):
        return self._format(a - b)

    def multiply(self, a, b):
        return self._format(a * b)

    def divide(self, a, b):
        if b == 0:
            raise ValueError("Division by zero")
        result = a / b
        if result.is_integer():
            return self._format(int(result))
        return str(result)   # keep float as decimal

class Menu:
    """Handles all user interaction with coloured output."""

    def __init__(self):
        self.calc = Calculator()

    def display_header(self):
```

```
        """Show a coloured header."""
        print(Fore.CYAN + Back.BLACK + "=" * 40)
        print(Fore.YELLOW + "      ADVANCED CALCULATOR")
        print(Fore.CYAN + "=" * 40 + Style.RESET_ALL)

    def show_main_menu(self):
        """Display the main menu options."""
        print(Fore.GREEN + "\n--- MAIN MENU ---")
        print(Fore.WHITE + "1. Add")
        print("2. Subtract")
        print("3. Multiply")
        print("4. Divide")
        print("5. Change output base")
        print("6. Exit")

    def show_base_menu(self):
        """Display base selection submenu."""
        print(Fore.MAGENTA + "\n--- BASE OPTIONS ---")
        print("1. Decimal")
        print("2. Octal")
        print("3. Binary")
        print("4. Back to main menu")

    def get_number(self, prompt):
        """Safely get a float from the user."""
        while True:
            try:
                return float(input(prompt))
            except ValueError:
                print(Fore.RED + "Invalid number. Please try again.")

    def get_choice(self, max_choice):
        """Get a valid menu choice."""
        while True:
            try:
                choice = int(input(Fore.YELLOW + "Your choice: " +
Style.RESET_ALL))
                if 1 <= choice <= max_choice:
                    return choice
                else:
                    print(Fore.RED + f"Please enter a number between 1 and
{max_choice}.")
            except ValueError:
                print(Fore.RED + "Invalid input. Enter a number.")

    def run(self):
        """Main program loop."""
        self.display_header()

        # Initial base selection
        print(Fore.CYAN + "Choose initial output base:")
        self.show_base_menu()
        base_choice = self.get_choice(3)
        if base_choice == 1:
```

```
            self.calc.base = "decimal"
        elif base_choice == 2:
            self.calc.base = "octal"
        else:
            self.calc.base = "binary"
        print(Fore.GREEN + f"Base set to {self.calc.base.upper()}")

        while True:
            self.show_main_menu()
            choice = self.get_choice(6)

            if choice == 6:
                print(Fore.BLUE + "Thank you for using the calculator. Goodbye!")
                break

            if choice == 5:
                # Change base
                self.show_base_menu()
                base_choice = self.get_choice(4)
                if base_choice == 4:
                    continue
                new_base = {1: "decimal", 2: "octal", 3: "binary"}[base_choice]
                try:
                    self.calc.base = new_base
                    print(Fore.GREEN + f"Base changed to
{self.calc.base.upper()}")
                except ValueError as e:
                    print(Fore.RED + str(e))
                continue

            # For operations 1-4, ask for two numbers
            print(Fore.CYAN + "\nEnter two numbers:")
            a = self.get_number("First number: ")
            b = self.get_number("Second number: ")

            try:
                if choice == 1:
                    result = self.calc.add(a, b)
                    op = "+"
                elif choice == 2:
                    result = self.calc.subtract(a, b)
                    op = "-"
                elif choice == 3:
                    result = self.calc.multiply(a, b)
                    op = "*"
                elif choice == 4:
                    result = self.calc.divide(a, b)
                    op = "/"

                # Colourful output
                print(Fore.LIGHTGREEN_EX + f"\nResult: {a} {op} {b} = ", end="")
                print(Fore.LIGHTRED_EX + f"{result} (in {self.calc.base})" +
Style.RESET_ALL)
            except ValueError as e:
```

```
                print(Fore.RED + f"Error: {e}")

def main():
    menu = Menu()
    menu.run()

if __name__ == "__main__":
    main()
```

What we added with `Menu` class:

- Encapsulation of all user-interface logic.
- Colour coding:
  - `Fore.GREEN` for menus
  - `Fore.RED` for errors
  - `Fore.CYAN` for headers
  - `Fore.LIGHTGREEN_EX` / `LIGHTRED_EX` for results
- Input validation methods (`get_number`, `get_choice`).
- Separation of concerns: `Calculator` does math, `Menu` handles I/O.

Full Code

You can copy the final code from **Step 4** – it includes the `Calculator` class with base property and the `Menu` class with colour.

How to Run

1. Save the final code in a file, e.g. `calculator_oop.py`.
2. Install colorama: `pip install colorama`
3. Run the script:

```
python calculator_oop.py
```

Summary of OOP Concepts Learned

Concept	Example in our code
Class	`class Calculator:` , `class Menu:`
Object / Instance	`calc = Calculator()`
Methods	`add()`, `subtract()`, `run()`
Attributes	`self._base`
Encapsulation	`_base` (internal), exposed via `@property`

Concept	Example in our code
Property getter/setter	`@property` and `@base.setter`
Separation of concerns	`Calculator` does math, `Menu` does UI

You have now built a complete OOP calculator with configurable base and a polished colour menu. Experiment by adding more operations (power, square root) or saving history – the OOP structure makes it easy to extend!